System Architecture Design

Web-Based Vehicle Auction Platform

---

1. Architectural Style

Three-Tier (Layered) Architecture

The system follows a **three-tier architecture**, which separates concerns into:

1. Presentation Layer (Frontend)
2. Application Layer (Backend / Business Logic)
3. Data Layer (Database)

This architecture is widely used in **enterprise web applications** due to its:

- Scalability
- Maintainability
- Security
- Clear separation of responsibilities

2. High-Level Architecture Overview

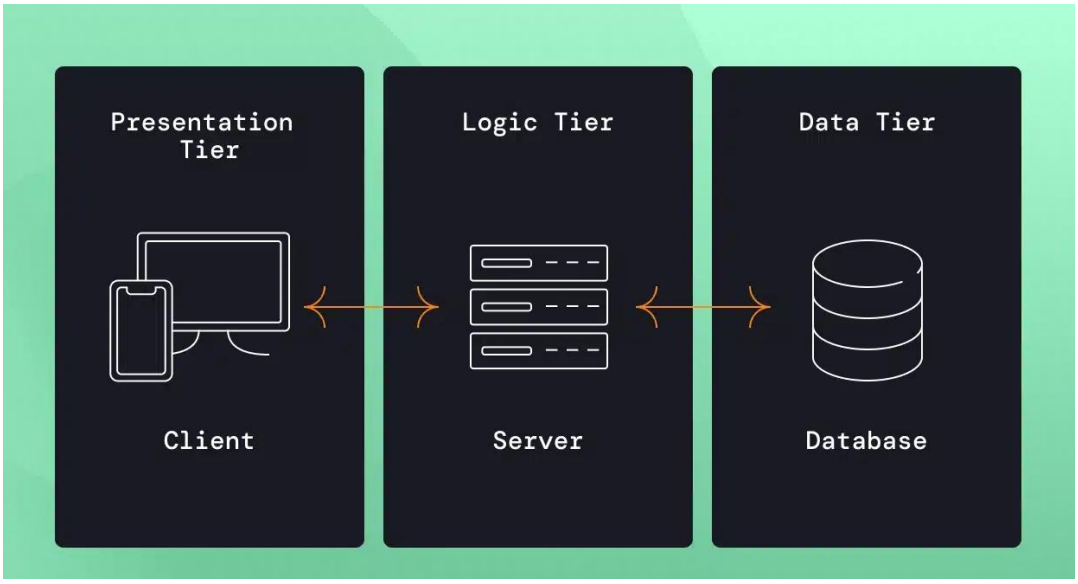


Figure 1 Three-tier architecture that will be maintained

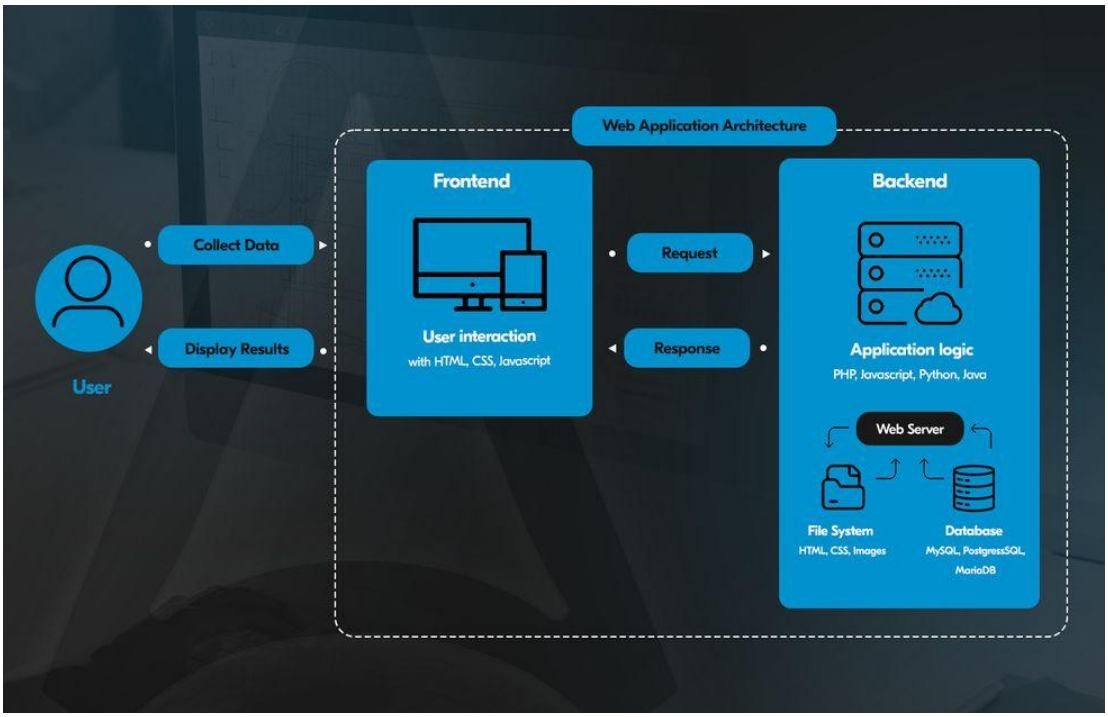


Figure 2 Detailed view of the three-tier architecture

3. Layer Descriptions

3.1 Presentation Layer (Frontend)

Responsibilities:

- User interaction
- Display vehicle listings and auction details
- Capture user inputs (registration, login, bids)
- Communicate with backend via HTTP/REST APIs

Users:

- Guest users
- Registered users
- Verified buyers
- Admins

Key Characteristics:

- No business logic
- No direct database access
- Role-based UI rendering (admin vs buyer)

3.2 Application Layer (Backend)

Responsibilities:

- Enforce business rules
- Handle authentication and authorization
- Manage auctions and bidding logic
- Validate requests
- Coordinate database operations

Core Functional Areas:

- User Management Service
- Vehicle Management Service
- Auction Management Service
- Bidding Service
- Admin Management Service

Key Characteristics:

- Stateless RESTful APIs
- Centralized security enforcement
- Transaction-controlled operations (especially bidding)
- Role-based access control

3.3 Data Layer (Database)

Responsibilities:

- Persistent storage of system data
- Maintain data integrity and consistency
- Support querying and reporting

Stored Data Includes:

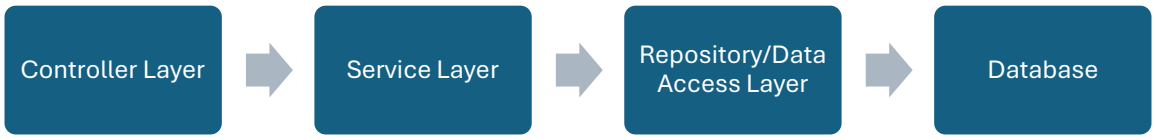
- Users and roles
- Vehicle listings
- Auctions
- Bids
- Audit timestamps

Key Characteristics:

- Relational structure
- Enforced constraints (foreign keys, uniqueness)
- ACID compliance

4. Component-Level Architecture (Backend Perspective)

Even at a high level, we define **logical components**.



This allows us to maintain:

- Clear responsibility separation
- Easy unit testing

5. Security Architecture Overview

Security is enforced centrally at the backend.

Security Concepts:

- Authentication: verifies user identity
- Authorization: controls access based on role

Role Model:

Role	Permissions
Guest	View vehicles
Registered User	View details
Verified Buyer	Place bids
Admin	Full control

This directly enforces:

- **FR-03**
- **BR-01**
- **BR-07**

6. Data Flow Summary (Conceptual)

1. User interacts with frontend
2. Frontend sends REST request
3. Backend validates:
    - Authentication
    - Authorization
    - Business rules
4. Backend reads/writes database
5. Response returned to frontend

This ensures:

- No direct DB exposure
- No rule bypass
- Full control over auction logic

7. Architectural Constraints

- No payment processing
- No external auction integrations
- No real-time socket requirement (initial version)
- Designed for moderate concurrent users

8. Architectural Quality Attributes

Attribute	How Architecture Supports It
Security	Centralized backend enforcement
Maintainability	Layer separation
Scalability	Stateless backend
Reliability	Transaction-safe operations
Testability	Clear component boundaries